

Algoritmos de ordenação avançados

① Ordenação por intercalação (Merge Sort)

Consiste em aplicar repetidas vezes o algoritmo de intercalação.

Intercalação: dado um vector $V[e \dots d]$ tal que $V[e \dots m-1]$ e $V[m \dots d]$ estejam ordenados, queremos que $V[e \dots d]$ esteja ordenado.

```
VOID INTERCALA (INT *V, INT e, INT m, INT d) {  
    INT i = e, j = m+1, k = 0;  
    INT *W = MALLOC((d - e + 1) * sizeof(INT));  
    WHILE (i <= m && j <= d)  
        IF (V[i] < V[j]) W[k++] = V[i];  
        ELSE W[k++] = V[j];  
    WHILE (i <= m) W[k++] = V[i++];  
    WHILE (j <= d) W[k++] = V[j++];  
    FOR (i = e, k = 0; i <= d, i++, k++) V[i] = W[k];  
    FREE(W);  
}
```

$O(d - e + 1)$ (linear)

0	1	2	3	4	5	6	7	
-2	0	2	3	-3	1	4	6	= V
e				m		j	d	
i	i	i	i	i	i	j		
INICIO	INCREMENTANDO			INICIO	INCREMENTANDO			

-3	-2	0	1	2	3	4	6	= w
0	1	2	3	4	5	6	7	

w =	-2	-1	0	1	2	3	4	7
-----	----	----	---	---	---	---	---	---

e = 0

d = 3

m = 1

w =	2	3
-----	---	---

w =	-1	2	3	7
			i =	

$$m = (0 + 7) / 2 = 3$$

nivel 0

0	1	2	3	4	5	6	7	
2	3	-1	7	-2	0	1	4	m = 8

8

0	1	2	3	4	5	6	7	
-1	2	3	7	-2	0	1	4	

$$m = (0 + 3) / 2$$

nivel 1

0	1	2	3	4	5	6	7	
2	3	-1	7	-2	0	1	4	

2

0	1	2	3	4	5	6	7	
3	2	7	-1	0	-2	1	4	1

$$\rightarrow 1 \times \frac{1}{1} O(n) = O(n)$$

1 vector de n elementos

$$1 \rightarrow \frac{n}{2^0}$$

$$\rightarrow 2 \times \frac{1}{2} O(n) = O(n)$$

2 vectores de $\frac{n}{2}$ elementos

$$2 \rightarrow \frac{n}{2^1}$$

$$\rightarrow 4 \times \frac{1}{4} O(n) = O(n)$$

4 vectores de $\frac{n}{4}$ elementos

$$4 \rightarrow \frac{n}{2^2}$$

$$\dots \quad 2^q \rightarrow \frac{n}{2^q} \leq 1$$

$$\rightarrow n \leq 2^q$$

$$\lg n \leq \lg 2^q \therefore q \geq \lg n$$

```
void MERGESORT(int *v, int e, int d){  
    if(e < d){  
        int m = (e + d) / 2;  
        MERGESORT(v, e, m);  
        MERGESORT(v, m + 1, d);  
        INTERCALA(v, e, m, d);  
    }  
}
```